WEEK-1

Name: N. Hema Harika

Roll no: CH.SC.U4CSE24231

1. Write a program to write sum of first n natural numbers using user defined function.

Code:

```
#include<stdio.h>
 int add(int n){
   int sum=0;
   int i;
   for(i=1;i<=n;i++){
     sum+=i;
   }
   printf("%d",sum);
 }
 int main(){
   int n;
   printf("Enter the n value:");
   scanf("%d",&n);
   add(n);
   printf("\n");
 }
```

Output:

```
root@amma43:/home/amma/Documents# gcc -o num num.c
root@amma43:/home/amma/Documents# ./num
Enter the n value:4
10
```

Space complexity: The function uses variables n, sum, and loop variable i. Space does not change with the value of n. so the space complexity for this code is **0(1).**

Time Complexity: The program runs a **loop from 1 to n**, doing one operation each time.Time complexity is **O(n).**

2. Write a program to find sum of squares of the first natural numbers.

Code:

```
#include<stdio.h>
int main(){
  int n;
  printf("Enter the value of n");
  scanf("%d",&n);
  int i;
  int sum=0;
  for(i=1;i<=n;i++){
    sum+=i*i;
  }
  printf("%d",sum);
}
```

Output:

```
root@amma43:/home/amma/Documents# gcc -o squares squares.c
root@amma43:/home/amma/Documents# ./squares

Enter the value of n 5
55
```

Space complexity: The function uses variables `n`, `sum`, and loop variable `i`. Space does not change with the value of n. so the space complexity for this code is **0(1).**

Time Complexity: The program runs a **loop from 1 to n** , doing one operation each time. Time complexity is **O(n).**

3.Write a program to find sums of the first natural numbers.

Code:

```
#include<stdio.h>
 int main(){
   int n;
   printf("Enter the value of n");
   scanf("%d",&n);
   int i;
   int sum=0;
   for(i=1;i<=n;i++){
     sum+=i*i*i;
   }
   printf("%d",sum);
 }
```

Output:

```
root@amma43:/home/amma/Documents# gcc -o cubes cubes.c
root@amma43:/home/amma/Documents# ./cubes
Enter the value of n:3
36
```

Space complexity: The function uses variables `n`, `sum`, and loop variable `i`. Space does not change with the value of n., so the space complexity for this code is **0(1).**

Time Complexity: The program runs a **loop from 1 ton** , doing one operation each time. Time complexity is **O(n).**

4. Write a program to write a factorial of a given integer using recursion.

Code:

```
#include<stdio.h>
  int fact(int n){
    if(n>1){
      return n*fact(n-1);
    }

  }
  int main(){
    int n;
    printf("Enter the number:");
    scanf("%d",&n);
    fact(n);
    int result=fact(n);
    printf("%d",result);
    printf("\n");
  }
```

Output:

```
root@amma43:/home/amma/Documents# gcc -o fact fact.c
root@amma43:/home/amma/Documents# ./fact
Enter the number:4
24
```

Space complexity: Factorial makes recursive calls before returning. Hence, it uses **O(n)** space complexity.

Time complexity: The function calls itself n times. Time complexity is **O(n).**

5. Write a program for transposing a 3x3 matrix.

Code:

```
#include<stdio.h>
  int main(){
    int a[3][3]={{1,3,6},{4,9,6},{2,6,4}};
    int i,j;
    for(i=0;i<3;i++){
      for(j=0;j<3;j++){
        printf("%d\t",a[j][i]);
      }
      printf("\n");
    }
  }
```

Output:

```
root@amma43:/home/amma/Documents# gcc -o matrix matrix.c
root@amma43:/home/amma/Documents# ./matrix
1       4       2
3       9       6
6       6       4
```

Space complexity: The function uses variables `n`, `sum`, and loop variable `i`. Space does not change with the value of n. space complexity for this code is **0(1).**

Time complexity: To transpose an **n** matrix, you must visit **every element at** once. This requires **two nested loops so** total operations = **$n \times n = n^2$. O(n^2) is Time complexity.**

6. Write a program to find the Fibonacci series.

Code:

```
#include<stdio.h>
  int main(){
    int a=0;
    int b=1;
    int c;
    int i;
    printf("%d\t",a);
    printf("%d\t",b);
    for(i=1;i<10;i++){
      c=a+b;
      printf("%d\t",c);
      a=b;
      b=c;
    }
    printf("\n");
  }
```

Output:

```
root@amma43:/home/amma/Documents# gcc -o fibonacci fibonacci.c
root@amma43:/home/amma/Documents# ./fibonacci
0       1       1       2       3       5       8       13      21      34      55
```

Space complexity: The program uses only a fixed number of variables. Space complexity is **O (1).**

Time complexity: The loop runs **n times**, printing one Fibonacci number in each iteration → **O(n)**.